

PENETRATION TESTING REPORT

Broken Access Control & Insecure Direct Object Reference (IDOR)

Target: DVWA (Damn Vulnerable Web Application) behind SafeLine WAF

Field	Detail
Report Title	Penetration Testing Report — Broken Access Control & IDOR
Test Type	Authenticated, black-box / grey-box web application penetration test
Methodology	OWASP Testing Guide (WSTG) — Authorization Testing
Target	DVWA v1.9/1.10, reverse-proxied by SafeLine WAF
Environment	Kali Linux (WSL2) -> Docker -> SafeLine WAF -> DVWA
Classification	Public / Educational - Lab Environment
Tester	andriyank
Report Date	23 July 2026

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Scope & Methodology	3
In Scope	3
Out of Scope	3
Methodology	3
Test Accounts & Assumptions	4
3. Environment Setup	4
4. Findings Summary	4
5. Detailed Findings	4
5.1 Forced Browsing - Missing Function-Level Access Control	5
5.2 IDOR - Insecure Direct Object Reference	5
6. Attack Narrative	6
7. WAF Detection Analysis	6
Why This Happens	7
Detection Coverage Summary	7
8. Overall Conclusion & Strategic Recommendations	7
Conclusion	7
Strategic Recommendations	8
9. References	8

1. Executive Summary

This engagement assessed whether a signature-based Web Application Firewall (SafeLine WAF) placed in front of a deliberately vulnerable application (DVWA) is sufficient to detect and stop Broken Access Control vulnerabilities, specifically Insecure Direct Object Reference (IDOR) and Forced Browsing.

Two test scenarios were executed against the Broken Access Control vulnerability class (OWASP Top 10 A01:2021):

- Function-level access control (forced browsing to an authenticated module while logged out) was correctly enforced by the application itself.
- Object-level access control (IDOR) was not enforced. An authenticated user (admin) was able to retrieve other users' personal data simply by changing a numeric id parameter in the URL, with no server-side ownership check.

The WAF did not detect or block the IDOR requests, because the requests were syntactically valid HTTP traffic with no attack-pattern signature to match against.

Overall Risk Rating: High. While the application correctly gates access at the page level, it fails to gate access at the record level - meaning any authenticated user, regardless of privilege, can enumerate and read other users' data. This class of vulnerability sits outside the detection capability of any purely signature/pattern-based WAF and must be fixed in application code.

Metric	Value
Total scenarios tested	2
Vulnerabilities confirmed	1 (IDOR)
Controls confirmed working	1 (Forced Browsing prevention)
Highest severity finding	High (IDOR)
WAF detection rate on findings	0% (IDOR not logged/blocked)

2. Scope & Methodology

In Scope

- <http://127.0.0.1:8080/> (DVWA, accessed through SafeLine WAF reverse proxy)
- Vulnerability class under test: OWASP Top 10 A01:2021 - Broken Access Control

Out of Scope

- Other DVWA modules/vulnerability classes (SQLi, XSS, Path Traversal were only used as WAF-detection control checks, not as primary findings)
- Underlying host, Docker engine, and SafeLine WAF management plane

Methodology

Testing followed the OWASP Web Security Testing Guide (WSTG) authorization-testing category, specifically:

- WSTG-ATHZ-02 - Testing for Bypassing Authorization Schema (forced browsing)
- WSTG-ATHZ-04 - Testing for Insecure Direct Object References

Test Accounts & Assumptions

DVWA default account admin / password, DVWA Security Level set to Low to establish a baseline (no application-side hardening).

This is a lab exercise using DVWA, a purposely vulnerable application. Findings illustrate a vulnerability class and WAF-coverage gap; severity ratings are expressed as if the same flaw existed in a production application handling real user data.

3. Environment Setup

```
Laptop (Windows 11)
  WSL2
    Kali Linux Rolling 2026.2
      Docker Engine
        SafeLine WAF (containers: mgt, tengine, detector, postgres, etc.)
          |
          +-- Listening Port: 8080 (HTTP)
          |
          +-- Reverse Proxy --> DVWA (port 9001)

sudo docker run -d --name dvwa -p 9001:80 vulnerables/web-dvwa
```

Setup steps:

- <http://127.0.0.1:9001/setup.php> -> Create / Reset Database
- Log in (admin / password) at <http://127.0.0.1:9001/login.php>
- Set DVWA Security -> Low
- SafeLine's upstream (Applications -> Test Web Demo -> Basic) re-pointed from <http://127.0.0.1:8000> to <http://127.0.0.1:9001>, exposing DVWA through the WAF at <http://127.0.0.1:8080>

4. Findings Summary

ID	Finding	CWE	Severity	Status
F-01	Forced Browsing to authenticated module while unauthenticated	CWE-862	N/A	Not Vulnerable
F-02	IDOR on id parameter - cross-user data disclosure	CWE-639, CWE-284	High	Vulnerable

5. Detailed Findings

5.1 Forced Browsing - Missing Function-Level Access Control

Attribute	Detail
CWE	CWE-862 (Missing Authorization)
Severity	N/A (control passed)
Location	/vulnerabilities/exec/

Description: Tested whether an unauthenticated user could reach an authenticated module directly by URL.

Steps to Reproduce / PoC:

1. Logout: <http://127.0.0.1:8080/logout.php>
2. Direct access without login: <http://127.0.0.1:8080/vulnerabilities/exec/>

Result: The request was automatically redirected back to login.php. DVWA enforces a session check on every module page, so unauthenticated access was successfully prevented by the application, independent of the WAF.

Impact: None - no unauthorized access achieved.

Remediation: No action required; continue enforcing server-side session/auth checks on every protected route. Recommend adding this check as a regression test to prevent future refactors from removing it.

5.2 IDOR - Insecure Direct Object Reference

Attribute	Detail
CWE	CWE-639 (Authorization Bypass Through User-Controlled Key), CWE-284
Severity	High
Estimated CVSS 3.1	6.5-7.5 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
Location	/vulnerabilities/sqli/?id={n}&Submit=Submit

Description: Logged in as admin, the id parameter of the SQLi module was changed sequentially. The application returned other users' records with no verification that the requested id belongs to the authenticated session.

Steps to Reproduce / PoC:

```
http://127.0.0.1:8080/vulnerabilities/sqli/?id=1&Submit=Submit
http://127.0.0.1:8080/vulnerabilities/sqli/?id=2&Submit=Submit
http://127.0.0.1:8080/vulnerabilities/sqli/?id=3&Submit=Submit
```

Result / Evidence:

Request	Data Returned
id=1	First name: admin, Surname: admin
id=2	First name: Gordon, Surname: Brown

Request	Data Returned
id=3	First name: Hack, Surname: Me

Object references (id) are direct, sequential, and easily guessable; ownership of the referenced record is never verified server-side.

Impact: Any authenticated user - regardless of role - can enumerate id values to harvest other users' personal data. In a production system storing PII, financial, or health data, this could constitute a large-scale data breach with legal/regulatory exposure (e.g., GDPR/UU PDP).

Likelihood: High - no special tooling required; a simple sequential loop over id is sufficient.

Remediation:

- Enforce server-side ownership checks on every object access (if requested_id != logged_in_user_id -> return 403 Forbidden)
- Use indirect/non-guessable object references (random UUIDs) instead of sequential integers
- Enforce Role-Based Access Control (RBAC) consistently on every endpoint that returns user-specific data, not just on page-level routing
- Add automated authorization tests (e.g., "user A cannot read user B's resource") to CI/CD to catch regressions

Priority: Immediate (critical path fix before production release).

6. Attack Narrative

To illustrate real-world impact, a plausible end-to-end scenario:

- Initial access: An attacker registers or obtains a low-privilege, legitimate account on the application (no special exploit needed - normal signup/login).
- Discovery: While using a normal feature (e.g., viewing their own profile/order/record), the attacker notices a numeric id parameter in the URL.
- Exploitation: The attacker iterates the id parameter (id=1, id=2, id=3, ...), either manually or with a simple script (e.g., curl/Burp Intruder loop), and the application returns other users' data for each valid id.
- Impact realized: The attacker mass-harvests personal data belonging to every other user in the system - a full account/data enumeration, achieved entirely through the application's front door, using only a single low-privilege session and no WAF-triggering payloads.
- Detection gap: Because every individual request is syntactically benign, none of it appears in the WAF's attack log, so the breach can proceed undetected for as long as the attacker chooses to run it.

This narrative underscores that IDOR is not a "low-noise" curiosity - at scale, it is a full data-breach vector that bypasses perimeter defenses entirely.

7. WAF Detection Analysis

SafeLine WAF - Attacks log for this test session:

Action	URL	Attack Type
Blocked	?file=../../../../../etc/passwd	File Include
Blocked	?id=1' OR '1'='1	SQL Injection
Blocked	?q=<script>alert(1)</script>	XSS
- (not logged)	?id=2&Submit=Submit	-
- (not logged)	?id=3&Submit=Submit	-

The id=2 and id=3 requests do not appear at all in the Attacks log - not because they were missed, but because syntactically the requests are entirely valid and contain no pattern recognized by signature-based detection.

Why This Happens

SafeLine WAF, like most signature-based WAFs, matches request patterns/payloads (SQL characters, script tags, ../ sequences, etc.) against a predefined ruleset. This is effective against attack classes like XSS, SQLi, and Path Traversal, whose payloads inherently contain recognizable structures. Broken Access Control and IDOR fall outside this detection scope because:

- The request is syntactically valid - ?id=2 is an ordinary numeric parameter with no suspicious characters.
- The flaw is in the application's authorization logic, not the request's shape.
- A WAF has no business-logic context to know "User A should not be able to view User B's data" - that responsibility belongs entirely to the application code.

Detection Coverage Summary

Attack Type	WAF Coverage
XSS	Effective
SQL Injection	Effective
Path Traversal	Effective
Forced Browsing	N/A - handled by app, not WAF
IDOR / Broken Access Control	Out of scope for signature-based WAF

8. Overall Conclusion & Strategic Recommendations

Conclusion

The tested environment demonstrates a common and important gap in defense-in-depth strategy. A signature-based WAF (SafeLine) provides strong, effective protection against classic injection-style attacks (XSS, SQLi, Path Traversal), and the application's own session/authentication gate correctly blocks unauthenticated access at the page level. However, object-level authorization (IDOR) is not - and

structurally cannot be - covered by a signature-based WAF. This is a business-logic flaw that must be remediated in application code, not compensated for at the network/perimeter layer.

Strategic Recommendations

- Do not rely on WAF as the sole control for Broken Access Control. Treat WAF as one layer in defense-in-depth, not a substitute for secure coding.
- Adopt secure-by-design object referencing across the codebase (UUIDs, ownership checks) as a standing engineering guideline, not a one-off patch.
- Introduce authorization-focused test cases into the SDLC (unit/integration tests simulating cross-user access attempts) so IDOR-class bugs are caught before release.
- Periodic manual/authenticated penetration testing - since IDOR requires an authenticated session and business-logic understanding, it will typically not be found by unauthenticated automated scanning or WAF logs alone.
- Consider a WAF/RASP combination or application-level authorization middleware for defense-in-depth if object-level access control cannot be guaranteed consistently across all endpoints.

9. References

- OWASP Top 10 - A01:2021 Broken Access Control - https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP - IDOR Prevention Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
- CWE-639: Authorization Bypass Through User-Controlled Key - <https://cwe.mitre.org/data/definitions/639.html>
- CWE-862: Missing Authorization - <https://cwe.mitre.org/data/definitions/862.html>
- DVWA - Damn Vulnerable Web Application (GitHub) - <https://github.com/digininja/DVWA>
- SafeLine WAF - Official Documentation - <https://waf.chaitin.com/>
- SafeLine WAF - GitHub - <https://github.com/chaitin/SafeLine>